NAME: K. HARSHIKA

REG .NO :192473020

COURSE CODE: CSA0609

Hospital Patient Record Sorting (Merge Sort)

A hospital system maintains patient records sorted by admission time for quick access and

reporting. Analyze how Merge Sort ensures stable and efficient sorting. Identify issues such

as large datasets and data consistency. Apply divide and conquer principles. Analyze time

complexity and performance. Design a solution and justify its use in healthcare systems.

Title

Hospital Patient Record Sorting Using Merge Sort

Hospitals are among the most data-intensive organizations in the world. Every day, they admit hundreds or even thousands of patients, each generating a unique medical record that contains information such as Patient ID, Name, Age, Gender, Admission Time, Department, Doctor Assigned, Diagnosis, Treatment Details, Laboratory Reports, Billing Information, and Discharge Status. These records are continuously updated throughout the patient's stay and are accessed by doctors, nurses, pharmacists, laboratory technicians, insurance providers, and hospital administrators.

In a large multi-specialty hospital, managing patient records manually is almost impossible because of the enormous volume of data generated every day. One of the most important requirements in a hospital information system is to organize patient records according to admission time. Sorting patient records chronologically helps medical professionals identify newly admitted patients quickly, prepare daily admission reports, monitor patient flow, allocate hospital beds efficiently, and prioritize emergency cases.

For example, suppose a hospital admits 2,000 patients in a single day. If patient records are stored in random order, doctors and administrators will spend significant time searching for recently admitted patients. During emergencies, even a small delay in accessing patient records can affect treatment decisions and patient safety.

The hospital database also continues to grow every year. Over time, millions of patient records may be stored. An inefficient sorting algorithm with quadratic time complexity becomes unsuitable because it increases processing time and system workload. Healthcare organizations therefore require an algorithm that offers consistent performance regardless of dataset size.

Merge Sort addresses these challenges by applying the Divide and Conquer technique. It divides the patient records into smaller groups, sorts them independently, and merges them into a single sorted list while maintaining data consistency.

Problem Statement

The hospital management system stores thousands of patient records every day. These records must be arranged according to admission time to ensure quick retrieval, efficient report generation, and accurate patient management. Traditional sorting methods become inefficient when the number of patient records increases significantly, leading to delays in searching, reporting, and decision-making.

The challenge is to design a sorting mechanism capable of handling large datasets while maintaining the correct order of patients who share the same admission time. The algorithm should minimize processing time, ensure stable sorting, provide predictable performance, and improve overall efficiency of hospital operations.

Therefore, this case study proposes the use of the Merge Sort algorithm to sort hospital patient records efficiently using the Divide and Conquer strategy.

Algorithm Used

Merge Sort Algorithm

Merge Sort is a comparison-based sorting algorithm that follows the Divide and Conquer paradigm. Instead of sorting the entire dataset directly, it repeatedly divides the dataset into smaller parts until each part contains only one element. These smaller lists are then merged together in sorted order to produce the final sorted list.

Working Principle

1. Divide the patient record list into two equal halves.
2. Continue dividing each half until only one patient record remains in each sublist.
3. Compare the admission times of patient records while merging.
4. Merge two sorted lists into one larger sorted list.
5. Repeat the process until the complete dataset becomes sorted.

Algorithm

MergeSort(A)

If size of A <= 1

  Return A

Find middle element

Divide A into Left and Right

MergeSort(Left)

MergeSort(Right)

Merge Left and Right

Return Sorted List

Merge Procedure

Merge(Left, Right)

Compare first elements

Place smaller element into output list

Move corresponding pointer

Repeat until one list becomes empty

Copy remaining elements

Return merged list

Justification for Choosing the Algorithm (Why Merge Sort?)

Merge Sort is particularly suitable for hospital information systems because it offers reliable performance, stability, and scalability. Healthcare databases often contain millions of patient records, and these records must be sorted efficiently without compromising data consistency.

One of the biggest advantages of Merge Sort is its guaranteed O(n log n) time complexity. Unlike Quick Sort, whose worst-case performance becomes $O(n^2)$, Merge Sort performs consistently even when the data is already sorted, reverse sorted, or randomly arranged. Hospitals require such predictable performance because delays in accessing patient records may directly affect patient care.

Another important reason for choosing Merge Sort is its stability. Suppose two patients are admitted at exactly 10:30 AM. If one patient registered first and another registered immediately afterward, their order should remain unchanged after sorting. Merge Sort preserves this original order, making it suitable for healthcare systems where chronological accuracy is essential.

Merge Sort is also highly efficient for large datasets stored in external memory. Since hospital databases often exceed available RAM, Merge Sort can efficiently process data stored on secondary storage devices.

Furthermore, Merge Sort divides the problem into smaller independent tasks, making it suitable for parallel processing in modern hospital servers.

The algorithm is also easier to analyze mathematically because its performance remains the same in all cases. This predictability makes it ideal for mission-critical healthcare applications.

Therefore, Merge Sort provides the best balance between efficiency, reliability, scalability, and stability for hospital patient record management.

Complexity Analysis

Time Complexity

Best Case

Even if the patient records are already sorted, Merge Sort still divides the list completely before merging.

Best Case = O(n log n)

Average Case

For randomly arranged patient records, Merge Sort performs logarithmic divisions and linear merging.

Average Case = O(n log n)

Worst Case

Unlike Quick Sort, Merge Sort never degrades to quadratic performance.

Worst Case = O(n log n)

Space Complexity

Merge Sort requires additional memory during the merge operation.

Space Complexity = O(n)

Complexity Table

Case	**Time Complexity**
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)

Manual Execution (Step-by-Step Process)

Original Patient Records

Patient ID	**Name**	**Admission Time**
P101	Ravi	10:30
P102	Priya	08:45

Case			Time Complexity
P103	Kiran	11:15	
P104	Anitha	09:20	

Step 1

Original List

[Ravi]

[Priya]

[Kiran]

[Anitha]

[Mohan]

[Sneha]

[Arjun]

[Divya]

Step 2

Divide

Left

Ravi

Priya

Kiran

Anitha

Right

Mohan

Sneha

Arjun

Divya

Step 3

Further Divide

Ravi Priya

↓

Ravi

Priya

Kiran Anitha

↓

Kiran

AnithaMohan Sneha

↓

Mohan

Sneha

Arjun Divya

↓

Arjun

Divya

Step 4

Merge Small Lists

Priya Ravi

Anitha Kiran

Sneha Mohan

Arjun Divya

Step 5

Merge Again

Priya

Anitha

Ravi

Kiran

Sneha

Arjun

Mohan

Divya

Step 6

Final Merge

Sneha

Priya

Anitha

Arjun

Mohan

Ravi

Kiran

Divya

Final Sorted Table

Patient ID	Name	Admission Time
P106	Sneha	08:15
P102	Priya	08:45
P104	Anitha	09:20
P107	Arjun	09:45
P105	Mohan	10:00
P101	Ravi	10:30
P103	Kiran	11:15
P108	Divya	11:30

Applications

Merge Sort is widely used in various healthcare and data-processing applications, including:

1. Hospital Information Systems (HIS)
2. Electronic Health Record (EHR) Management
3. Patient Admission Management
4. Emergency Department Queue Management
5. Laboratory Information Systems
6. Pharmacy Inventory Management
7. Insurance Claim Processing
8. Medical Billing Systems
9. Healthcare Data Analytics
10. Medical Research Databases
11. Government Health Record Management
12. Public Health Surveillance
13. Clinical Trial Data Processing
14. Radiology Record Management
15. Appointment Scheduling Systems

Advantages

1. Stable sorting algorithm.
2. Maintains admission order for equal timestamps.
3. Guaranteed O(n log n) performance.
4. Efficient for large datasets.
5. Suitable for linked lists.
6. Works efficiently with external storage.
7. Predictable execution time.
8. Parallel processing friendly.
9. Reliable performance.
10. High scalability.
11. Excellent for database systems.
12. Easy mathematical analysis.
13. Reduces search time after sorting.
14. Improves healthcare workflow.

15. Supports big data applications.

Limitations

1. Requires additional memory.
2. Not an in-place sorting algorithm.
3. Slightly slower than Quick Sort for very small datasets.
4. Recursive implementation increases function call overhead.
5. More memory consumption than Heap Sort.
6. Complex implementation compared to Bubble Sort.
7. Not ideal when memory is limited.
8. Frequent merging operations increase memory access.
9. Additional copying of data is required.
10. Less cache-friendly than Quick Sort.

Comparison with Other Algorithms

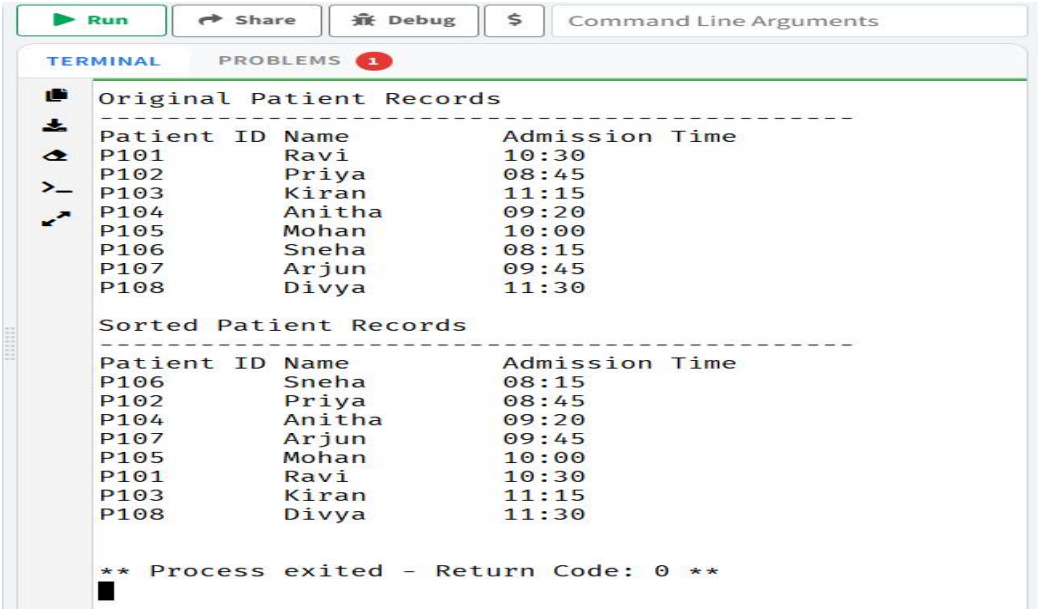
Algorithm	Best	Average	Worst	Stable	Extra Space	Suitable for Hospital Records
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	O(1)	Poor
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	O(1)	Poor
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	O(1)	Good for small datasets
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	No	O(log n)	Moderate
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No	O(1)	Good
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes	O(n)	Excellent

CODE

```
# Hospital Patient Record Sorting using Merge Sort
def merge_sort(records):
    if len(records) <= 1:
        return records
    mid = len(records) // 2
    left = merge_sort(records[:mid])
    right = merge_sort(records[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i][2] <= right[j][2]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Patient Records
```

```
patients = [
    ("P101", "Ravi", "10:30"),
    ("P102", "Priya", "08:45"),
    ("P103", "Kiran", "11:15"),
    ("P104", "Anitha", "09:20"),
    ("P105", "Mohan", "10:00"),
    ("P106", "Sneha", "08:15"),
    ("P107", "Arjun", "09:45"),
    ("P108", "Divya", "11:30")
]
print("Original Patient Records")
print("-" * 45)
print("{:<10} {:<12} {}".format("Patient ID", "Name", "Admission Time"))
for patient in patients:
    print("{:<10} {:<12} {}".format(patient[0], patient[1], patient[2]))
sorted_patients = merge_sort(patients)
print("\nSorted Patient Records")
print("-" * 45)
print("{:<10} {:<12} {}".format("Patient ID", "Name", "Admission Time"))

for patient in sorted_patients:
    print("{:<10} {:<12} {}".format(patient[0], patient[1], patient[2]))
```

OUTPUT


Run
Share
Debug
$
Command Line Arguments
TERMINAL
PROBLEMS 1
Original Patient Records
Patient ID Name Admission Time
P101 Ravi 10:30
P102 Priya 08:45
P103 Kiran 11:15
P104 Anitha 09:20
P105 Mohan 10:00
P106 Sneha 08:15
P107 Arjun 09:45
P108 Divya 11:30
Sorted Patient Records
Patient ID Name Admission Time
P106 Sneha 08:15
P102 Priya 08:45
P104 Anitha 09:20
P107 Arjun 09:45
P105 Mohan 10:00
P101 Ravi 10:30
P103 Kiran 11:15
P108 Divya 11:30
** Process exited - Return Code: 0 **